

手写 `vector`:从容量与构造分离到异常安全

CTC Practical C++ · 技术报告 · 2026-09-15

这份报告记录一次完整的 `my::vector<T>` 实现过程。题目来自 CTC 的 C++ 电面(60 分钟,与 `unique_ptr` 同场),要求覆盖 rule of five、完美转发、几何增长、容量与构造分离、搬迁判据、异常安全。

这道题不考算法。它考的全部是语言机制的边界:所有权、对象生命周期、值类别、异常传播路径。写业务代码十年可能一次都碰不到,写库的人天天在上面。

1 核心机制:容量与构造分离

整道题的地基是一句话:`vector` 必须能在"已分配但未构造"的内存上工作。

原因是 `capacity` 可以大于 `size`。那些多出来的槽位是真实存在的内存,但上面不能有对象——否则 `reserve(1000)` 会凭空构造一千个对象,而且 `T` 没有默认构造函数时直接编不过。

所以 `new T[n]` 这条路从一开始就被堵死了:它把分配和构造绑在一起。要拆开,需要四个操作,配成两对。

操作	做什么	不做什么
<code>::operator new(n)</code>	分配 <code>n</code> 字节原始内存	不调用任何构造函数
<code>new (p) T(args)</code>	在地址 <code>p</code> 上构造对象	不分配内存
<code>p->~T()</code>	析构单个对象	不释放内存
<code>::operator delete(p)</code>	释放原始内存	不调用任何析构函数

关键后果

RAII 在这里被手动打破了。`new/delete` 表达式帮你做的"分配+构造 / 析构+释放"配对,现在全部落到你手上。

凡是 `placement new` 构造过的对象,必须有一次对应的显式析构。这个配对关系是后面所有异常安全讨论的前提。

验证这条是否做到,有一个零成本的测试:让 `T` 没有默认构造函数。

```
struct NoDefault { int v; explicit NoDefault(int a) : v(a) {} };\n\nmy::vector<NoDefault> v;\nv.reserve(100);      // 用 new T[n] 的实现,这一行编不过
```

2 `reserve` 是唯一的资源迁移点

五个函数需要扩容能力:`push_back` 两个重载、`emplace_back`、`resize`、以及 `reserve` 自己。如果各写各的,五份重复的分配—搬迁—析构—释放逻辑,五个出错的地方。

收敛之后的调用链只有两层:

```
push_back(const T&) └─┐\npush_back(T&&)      └─┐─> emplace_back(Args&&...) └─┐\n                        └─┐─> resize(n) ─────────┐─> reserve(n)\n                        |                           |\n                        分配 / 搬迁 / 析构旧的 / 释放旧内存 <┐
```

真正需要动脑的只有 `reserve` 和 `emplace_back` 两个函数,其余是一行转发。

语义边界

题面必须写清楚,否则实现者只能按自己的假设走:

- `n <= capacity()` → 无操作。不重新分配,迭代器不失效。
- `n > capacity()` → 重新分配,搬迁全部元素,所有迭代器失效。
- 任何情况下 `size()` 和元素内容不变。

因为 `size_ <= capacity_` 恒成立,`n < size_` 必然落在"无操作"那一档。`reserve` 只管容量:缩容量是 `shrink_to_fit`,改元素个数是 `resize`,三个函数职责不重叠。

3 搬迁判据: `move_if_noexcept`

扩容时把旧元素搬到新缓冲区,用 `move` 还是 `copy`?

直觉上应该一律 `move`——旧对象搬完就要析构,搬空它不影响任何人。但这个直觉在异常面前失效:

反例

100 个元素,move 到第 50 个时移动构造抛了异常。此时前 49 个元素已经从旧缓冲区被掏空,新缓冲区只有 49 个。你无法把 `vector` 恢复到调用前的状态——原始数据已经不在任何一个完整的地方了。

如果走 `copy`,旧缓冲区始终完好,异常发生时把新缓冲区清理掉,`vector` 一字未动。

所以判据是:移动构造不抛异常时用 `move`,否则用 `copy`。标准库把这个判据封装成 `std::move_if_noexcept`,判断在编译期完成,无运行时开销。

```
new (mem + i) T(std::move_if_noexcept(ptr_[i]));
```

它的行为:

T 的性质	返回	搬迁方式
移动构造 <code>noexcept</code>	<code>T&&</code>	<code>move</code>
移动构造可能抛,且可拷贝	<code>const T&</code>	<code>copy</code> (安全优先)
移动构造可能抛,但不可拷贝	<code>T&&</code>	<code>move</code> (别无选择)

这就是 NOEXCEPT 不是装饰的原因

你的移动构造函数不标 `noexcept`,外层容器扩容时就会退化成 $O(n)$ 的深拷贝,而不是 $O(1)$ 的指针接管。这是行为差异,不是风格问题。

一处容易混淆的地方

`move_if_noexcept` 只属于扩容搬迁。`push_back(T&&)` 里应该是无条件 `std::move`:

```
void push_back(T&& val) {  
    // ...  
    new (ptr_ + size_) T(std::move(val)); // 不是 move_if_noexcept  
}
```

差别在于责任归属。扩容搬迁的源对象是 `vector` 自己管理的元素,搬坏了是容器的责任;而 `push_back(T&&)` 的实参

是调用方用右值主动交出来的,所有权已经转移,无条件搬走才是正确语义。

4 异常安全

reserve:强保证

需要保护的只有构造循环。`::operator new` 放在 `try` 外面——它抛的时候内存根本没拿到, `ptr_`、`size_`、`capacity_` 一个都没动,没有需要回滚的东西。

```
T* mem = static_cast<T*>(::operator new(sizeof(T) * val));
std::size_t i = 0;
try {
    for (; i < size_; i++) {
        new (mem + i) T(std::move_if_noexcept(ptr_[i]));
    }
}
catch (...) {
    for (std::size_t j = 0; j < i; j++) { mem[j].~T(); }
    ::operator delete(mem);
    throw;
}
```

三个细节:

- **i 声明在 try 外面。** `catch` 需要知道已经构造了几个,声明在 `for` 内部就拿不到这个边界。
- **先逐个析构,再释放内存。** `::operator delete` 不调析构;少了那个循环,已构造对象持有的堆资源全漏。
- **catch (...) 配 throw;。** `T` 的构造函数可以抛任意类型,不一定继承 `std::exception`。`throw e;` 会抛出一份被切片的拷贝,丢失派生类型信息;不带操作数的 `throw;` 重抛原对象。

push_back:强保证,但有一处松动

`push_back` 是 `reserve` 之后再加一步 placement new。那一步抛异常时,对象没构造成功, `++size_` 也没执行——`size_` 仍指向构造前的元素个数,那个失败的槽位不算在内,状态一致。

但严格的强保证要求 "no effects",而扩容已经发生: `capacity_` 变了,旧迭代器全部失效。

取舍

标准库的做法是**先在新缓冲区构造新元素,再搬迁旧元素**。新元素构造抛异常时,新缓冲区整块丢弃, `vector` 完全没被触碰,达到完整的 no effects。代价是实现顺序更绕。

本实现选择先扩容再构造,达到的是:**抛出后 `size()` 和已有元素不变、无泄漏,但 `capacity` 可能已增大、迭代器已失效**。面试里把这句话说出来就行,不必假装是完整的 no effects。

不需要 try/catch 的地方

```
try {
    new (ptr_ + size_) T(val);
    size_++;
}
catch (...) { throw; }    // 空操作,删掉
```

`catch` 里没有清理动作,只是原样重抛——不写这个 `try/catch`,行为完全相同。异常安全的判据是"有没有自己制造的、需要回滚的状态",不是"有没有可能抛"。

析构函数的异常

析构默认 `noexcept`, 抛出直接 `std::terminate`。即使显式写 `noexcept(false)` 解除, 栈展开期间再抛异常仍然 `terminate`——而 `reserve` 析构旧元素正好可能在清理路径上。

结论: 这不是你能处理的路径, 所以不进 `try` 块。标准库的契约直接要求 `T` 的析构不抛, 责任划给 `T` 的作者。

5 Rule of five 的四个陷阱

移动构造 / 移动赋值: 接管, 不是搬运

一个常见的错误实现是走 `reserve` + 逐个 `move`: 结果正确, 但代价差了一个数量级—— $O(n)$ 次移动构造加 n 个旧对象等着析构, 而且会分配内存、会抛, 标不了 `noexcept`。

正确做法是 $O(1)$ 的指针接管:

```
vector(vector&& other) noexcept {
    ptr_ = other.ptr_;
    size_ = other.size_;
    capacity_ = other.capacity_;
    other.ptr_ = nullptr;
    other.size_ = 0;           // 三个都要置空
    other.capacity_ = 0;
}
```

为什么三个成员都要置空

只置 `ptr_` 不够。其他析构时循环是 `for (i = 0; i < size_; ++i) ptr_[i].~T();`——`size_` 还是 5 而 `ptr_` 是 `nullptr`, 第一次迭代就解引用空指针。

即使在析构里加空指针判断挡住崩溃, `other.size()` 仍然返回 5、`empty()` 返回 `false`, 对象在说谎。

移动后的源对象状态是 **valid but unspecified**: 可以析构、可以赋新值、可以 `clear()`, 只是不该读它原来的值。不是“不合法”, 也不是“不能用”。

自赋值检查: 属于赋值, 不属于构造

`if (this != &other)` 在赋值运算符里是承重的, 在构造函数里是死代码——正在构造的对象不可能就是 `other`。更糟的是, 如果那个不可能的条件为假, 三个成员全是未初始化值。

移动赋值里它必须在:

```
vector& operator=(vector&& other) noexcept {
    if (this != &other) {
        clear();                // 销毁自己的元素
        ::operator delete(ptr_); // 释放自己的内存
        ptr_ = other.ptr_; /* ... 接管 ... */
        other.ptr_ = nullptr; /* ... 置空 ... */
    }
    return *this;
}
```

没有这个检查, `v = std::move(v)` 会先把自己的资源全毁掉, 然后从已释放的内存里读指针。

一个常见的误解

"`v = std::move(v)` 是 UB,所以不用管"——不成立。这句代码本身是合法的,编译器只给 `-Wself-move` 警告。UB 是 *你的实现* 制造出来的,不是语言规定的。标准对标准库类型的要求是 `valid but unspecified`;对你自己写的类,行为由你决定。

copy-and-swap:另一条路

拷贝赋值有一个更短的写法:

```
vector& operator=(const vector& other) {  
    vector tmp(other);  
    swap(tmp);  
    return *this;  
}
```

复用拷贝构造, `swap` 只换三个成员, `tmp` 析构时带走旧数据。天然自赋值安全、天然强异常保证。代价是多一次分配。两种写法都能过,取舍看场景。

6 const 正确性:哪些要两份

判据不是"这个函数改不改对象",而是返回值有没有暴露内部。

函数	返回	需要几份	原因
<code>size()</code> <code>capacity()</code> <code>empty()</code>	值	一份 <code>const</code>	拿到的是拷贝,改不了容器
<code>operator[]</code> <code>at()</code>	引用	两份	引用能写回容器
<code>begin()</code> <code>end()</code>	指针	两份	指针能写回容器

非 `const` 对象可以调 `const` 成员函数,所以返回值的函数只要一份;返回引用或指针的必须成对——`const` 对象上拿到 `const T&` / `const T*`,非 `const` 对象上拿到 `T&` / `T*`。

顺带一提:`range-for` 对迭代器的要求只有 `!=`、前置 `++`、一元 `*`,裸指针三个都内建。`T*` 完全够用,包一个 `iterator` 类是为了类型安全,不是语法要求。

7 错误分类

整个实现过程中出现的错误,去掉编译器能直接抓的 `typo`,实质性的只收敛到四类:

#	类别	典型表现
1	接管资源后源对象没置空	移动构造只置 <code>ptr_</code> ,漏掉 <code>size_ / capacity_</code> ;double free
2	释放内存时对象没析构	<code>catch</code> 里直接 <code>::operator delete(mem)</code> ,已构造对象的资源全漏
3	异常路径上自己申请的东西没回滚	<code>catch</code> 里 <code>break</code> 吞掉异常; <code>resize</code> 扩大分支无保护
4	<code>const</code> / <code>noexcept</code> 漏标	<code>begin()</code> 只有 <code>const</code> 版返回 <code>T*</code> ;移动构造不标 <code>noexcept</code>

另有两个只在模板里出现的坑,值得单独记:

- **模板成员函数不调用就不实例化**。数组特化的移动赋值里一个 `return ptr;` (类型完全不匹配)可以潜伏过整个测试套件,只要没人调过它。自测时每个成员函数至少要调一次,或者用 `template class my::vector<int>;` 强制实例化全部成员。

- 指针算术的隐式缩放。`mem + offset` 里编译器已经乘过 `sizeof(T)`,再写 `offset += sizeof(T)` 就是乘了两次。

8 验证

最终实现通过 1121 个检查点,ASan/UBSan 干净, `-Wall -Wextra` 零警告。覆盖范围:

- 空 `vector`、`push_back` 左右值分流、`emplace_back` 就地构造与完美转发(左值/右值路由)
- `operator[]` / `at()` 的 `const` 与非 `const`、越界抛 `std::out_of_range`
- `reserve` 不构造任何对象、`resize` 双向、`clear` 保留 capacity
- 几何增长:1000 次 `push_back` 仅 11 次重新分配
- rule of five 全套,含自赋值与自移动赋值、移动后源对象可复用
- 无默认构造类型、move-only 类型
- 扩容前后迭代器失效判定
- `move_if_noexcept` 的 move/copy 分流(两个探针类型分别验证)
- `push_back` 异常回滚:拷贝构造在搬迁中途抛出,size 与元素不变、存活对象数零变化
- 200×5 轮 push/pop churn 无泄漏、`std::string` 全流程

9 面试要点

60 分钟里(还要加上 `unique_ptr`)写满 180 行不现实。按打分权重排:

1. **`reserve` 和 `emplace_back` 必须写**。容量与构造分离、完美转发、异常回滚三个考点全在这两个函数里。
2. **rule of five 必须写**,尤其移动那两个——接管而非搬运、三成员置空、自赋值检查、`noexcept`。
3. `push_back` 两行转发,`pop_back`、访问器几行带过。
4. `resize` 等写不完的,口头说明逻辑即可。

几个高概率追问,答案要能脱口而出:

- 为什么不能用 `new T[n]`?
- 扩容搬迁什么时候 move 什么时候 copy,判据是什么?
- `push_back` 提供哪一级异常安全,你的实现真达到了吗?
- 为什么 `vector` 有两个 `push_back` 却只有一个 `emplace_back`?(`push_back(T&&)` 里 `T` 是类模板参数,已确定,是右值引用;`emplace_back(Args&&...)` 里 `Args` 是函数模板参数,推导时才定,是转发引用)
- 增长因子为什么是 2,有人用 1.5 是为什么?(2 倍时前面所有已释放块的总和永远差一点够下一次用;1.5 倍下累积的空闲块最终能覆盖新请求,前提是分配器能合并相邻空闲块)

附: `::operator new` 只保证 `__STDCPP_DEFAULT_NEW_ALIGNMENT__` (常见为 16 字节)。`T` 是 over-aligned 类型(如 `alignas(64)`)时需要 C++17 的 `std::align_val_t` 重载。不在本题要求内,但在低延迟场景里 cache-line 对齐的类型很常见。